P.Vishwa Nandan(192473015)

TASK 7 – Override and Specialise Behaviour

Marks: 4

This task is about **Method Overriding in Java**. We use method overriding when different vehicle types need different behaviour.

The assignment asks us to explain:

1. Methods overridden in subclasses.
2. @Override annotation and overriding rules.
3. Use of the final keyword.
4. Overriding toString(), equals() and hashCode().
5. Alternative approach using switch statements and complexity analysis. CO1 JAVA

1. Methods Overridden in Subclasses

Method overriding means defining a method in a child class that already exists in the parent class with the same method signature.

In our parking system, different vehicles have different parking rates.

For example:

- TwoWheeler: Rs. 10 per hour.
- Car: Rs. 30 per hour.
- ElectricCar: Rs. 30 per hour.

Instead of using the same parking rate for all vehicles, we override the ratePerHour() method in subclasses.

Example:

```
abstract class Vehicle {


   String regNo;


   Vehicle(String regNo) {
      this.regNo = regNo;
   }
```

```
    abstract double ratePerHour();

    double calculateFee(int hours) {
        return hours * ratePerHour();
    }
}

class TwoWheeler extends Vehicle {

    TwoWheeler(String regNo) {
        super(regNo);
    }

    @Override
    double ratePerHour() {
        return 10;
    }
}

class Car extends Vehicle {

    Car(String regNo) {
        super(regNo);
    }

    @Override
    double ratePerHour() {
        return 30;
    }
```

}

Explanation:

- Vehicle is the parent class.
- TwoWheeler and Car are child classes.
- Both subclasses provide their own implementation of ratePerHour().
- The correct parking fee is calculated according to the vehicle type.

2. @Override Annotation and Rules of Overriding

The @Override annotation tells Java that a method is intended to override a parent class method.

Example:

```
class Vehicle {


   void display() {
      System.out.println("Vehicle details");
   }
}


class Car extends Vehicle {


   @Override
   void display() {
      System.out.println("Car details");
   }
}
```

Rules of Method Overriding:

1. The method name must be the same.
2. The parameter list must be the same.
3. The return type must be the same or a compatible covariant return type.
4. The child class cannot reduce the access level of the overridden method.

5. Static, private and final methods cannot be overridden.
6. The @Override annotation helps identify incorrect method declarations.

Method overriding allows subclasses to provide specialised implementations of parent class methods.

3. Use of final Keyword

The final keyword is used to restrict changes in Java.

It can be used with variables, methods and classes.

A. Final Variable

A final variable cannot be reassigned after initialization.

Example:

final String regNo = "TN01AB1234";

In our parking system, the vehicle registration number can be declared final because it should not change after the vehicle object is created.

B. Final Method

A final method cannot be overridden by subclasses.

Example:

```
class Vehicle {

   final void displayRules() {
      System.out.println("Follow parking rules");
   }
}
```

This ensures that subclasses cannot change the common parking rules.

C. Final Class

A final class cannot be inherited by another class.

Example:

```
final class ParkingReceipt {

   void display() {
      System.out.println("Parking Receipt");
```

```
    }
}
```

A final class is useful when inheritance should be prevented.

However, the Vehicle class should not be final because other vehicle classes must inherit from it.

4. Overriding toString(), equals() and hashCode()

Java provides three important methods in the Object class.

We can override these methods to define how Vehicle objects are displayed and compared.

A. toString()

The toString() method returns a string representation of an object.

Example:

```
class Vehicle {

    String regNo;

    Vehicle(String regNo) {
        this.regNo = regNo;
    }

    @Override
    public String toString() {
        return "Vehicle Registration Number: " + regNo;
    }
}
```

Output:

Vehicle Registration Number: TN01AB1234

It helps display vehicle information clearly.

B. equals()

The equals() method compares two objects.

By default, it checks whether two references represent the same object.

We override it to compare vehicle registration numbers.

Example:

```
@Override
public boolean equals(Object obj) {

   if (obj instanceof Vehicle) {
      Vehicle v = (Vehicle) obj;
      return regNo.equals(v.regNo);
   }

   return false;
}
```

If two Vehicle objects have the same registration number, the method returns true.

C. hashCode()

The hashCode() method returns an integer hash value for an object.

When two objects are equal according to equals(), they must have the same hash code.

Example:

```
@Override
public int hashCode() {
   return regNo.hashCode();
}
```

This is useful when storing Vehicle objects in collections such as HashSet and HashMap.

Why are all three methods important?

- toString() displays vehicle information.
- equals() compares vehicles using registration numbers.
- hashCode() supports correct object handling in hash-based collections.